

VisualInertialOdometryV3

Düzeltilme ve İyileştirme Raporu

Bu rapor, saf monoküler Görsel Odometri (VO) pipeline'ından gerçek bir IMU + Ceres sliding-window Bundle Adjustment tabanlı Görsel-İnertiyel Odometri (VIO) sistemine dönüşüm sürecinde bulunan ve düzeltilen tüm hatalarla, yapılan performans ve kod kalitesi iyileştirmelerini — BAŞARILI ve BAŞARISIZ denemeler dahil — kronolojik sırayla özetler. Her madde: sorun, neden oldu, ne yapıldı, ölçülen sonuç yapısında sunulmuştur.

Kapsam:

- Adım 1 — Kamera merkezi zincirleme hatası
- Adım 2 — Tam VIO entegrasyonu (IMU preintegration + Ceres sliding-window BA)
- Adım 3 — Build taşınabilirliği ve kamera distorsiyon düzeltilmesi
- Performans iyileştirmeleri (ORB, arama karmaşıklığı, çizim throttling)
- Görselleştirme / ATE hizalama düzeltmeleri
- Kritik hata avı: hız/bias/ölçek patlaması tespiti ve kalıcı çözümü
- Güncel durum ve dürüst değerlendirme (nFrames=800 vs 2000)
- Adım 4 (deneme, geri alındı) — KLT optik akış tabanlı izleme
- Adım 5 (deneme, geri alındı) — Çok-görüşlü (N-view) triangülasyon
- Genel özet tablosu ve sonraki adım: keyframe/paralaks tabanlı pencereleme

Adım 1 — Kamera Merkezi Zincirleme Hatası

Dosya: src/main.cpp

Sorun:

processFrames() içinde görelî pozlar (recoverPose'dan) $\text{chainedT} = R \cdot \text{chainedT} + t$ şeklinde doğru zincirleniyordu, fakat bu chainedT dünya→kamera dönüşümünün (extrinsic) öteleme bileşeniydi — kameranin gerçek dünya konumu DEĞİLDİ. Doğrudan bu değer çizildiği için yörünge döndürülmüş/çarpık görünüyordu.

Neden oldu:

cv::recoverPose standart konvansiyonda $X_{\text{cam}} = R \cdot X_{\text{world}} + t$ döndürür (world→camera). Kameranin gerçek konumu ise $C = -R^T \cdot t$ formülüyle geri çıkarılmalıdır; bu adım atlanmıştı.

Ne yapıldı:

Çizime beslenen değer $\text{cameraCenter} = -\text{chainedR.t()} * \text{chainedT}$ olarak düzeltildi. Bu formül, sonraki tüm adımlarda (main.cpp ve pip04_VioWindow.hpp'deki cameraCenterFromPose) tutarlı şekilde yeniden kullanıldı.

Sonuç:

Derleme doğrulandı (proje bu makinede ilk kez derlendi — OpenCV 4.5.4/Eigen3/Ceres 2.0.0 kurulu olduğu teyit edildi). Kod tabanındaki 'voRotationDeg=-75' gibi kalıntı görsel hilelerin bu temel hatayı gözle telafi etmeye çalıştığı, Adım 2 sonrası doğrulandı.

Adım 2 — Tam VIO Entegrasyonu

Proje daha önce IMU/BA dosyaları (Imupreintegration.hpp, Imufactor.hpp, Viobundleadjustment.hpp) yazılmış ama main.cpp'den hiç çağrılmıyordu — saf monoküler VO çalışıyordu (essential matrix'ten gelen t ölçeeksizdi). Bu adımda bu parçalar gerçek bir sliding-window VIO pipeline'ına bağlandı.

2.1 — Yol Boyunca Bulunan İki Bağımsız Matematik Hatası

Hata A — Konum residual'ı t_{wc} 'yi kamera merkezi sanıyordu (Imufactor.hpp)

Konum residual'ı pose[3..5]'i (world→camera extrinsic öteleme, ReprojectionError'daki ile aynı) doğrudan 'kamera konumu' gibi kullanıyordu. IMU fiziksel olarak GERÇEK dünya konumunu ($C = -R^T t$) integre eder — t 'yi değil. Düzeltme: residual hesabından önce $C_i = -R_i^T \cdot t_i$, $C_j = -R_j^T \cdot t_j$ açıkça hesaplanıp $C_j - C_i$ kullanıldı.

Hata B — IMU ölçümleri hiç kamera çerçevesine döndürülüyordu

ImuPreintegrator ham IMU verisini (gövde/IMU çerçevesi) doğrudan integre ediyordu, ama optimize edilen pose kamera çerçevesindeydi. R_{BS} (kamera↔gövde rotasyonu) kimlikten çok uzak ($\sim 90^\circ$) olduğundan bu bir tutarsızlıktı. Düzeltme: yeni rotatImuToCameraFrame() fonksiyonu ham ivme/gyro örneklerini $R_{\text{CB}} = R_{\text{BS}}^T$ ile kamera çerçevesine döndürüyor; ImuPreintegrator artık bu döndürülmüş veriyle çalışıyor.

2.2 — Statik Yerçekimi Kalibrasyonu

Sabit (0,0,9.81) varsayımı yerine, sekans başındaki ilk 200 (durgun varsayılan) ivmeölçer örneği ortalanıp normalize edilerek gerçek yerçekimi yönü kestirildi: $\text{gravity} = -\text{normalize}(\text{mean}(\text{accel}[0..200])) \cdot 9.81$. Ölçülen sonuç: yön (0.147, 9.092, 3.681), $\text{norm}=9.81$ — sabit varsayımdan belirgin şekilde farklı, kalibrasyonun çalıştığını doğruluyor.

2.3 — Yeni Bileşenler (pip04_VioWindow.hpp)

- buildFullCoverageTracks — ardışık kare eşleşmelerini zincirleyip pencerenin TAMAMINDA hayatta kalan izleri kurar
- triangulateAndFilterWindowTracks — IMU-seed'lenmiş pozlarla landmark triangülasyonu + cheirality filtresi
- seedWindowStatesFromImu — pencere başlangıç durumlarını IMU dead-reckoning ile doldurur (world→camera parametrizasyonuna göre doğru türetilmiş: $R[j]=\Delta R^T \cdot R[j-1]$)
- cameraCenterFromPose / projectionMatrixFromPose — Adım 1'in formülünün yeniden kullanılabilir hali

Pencere stratejisi: windowSize=5, pencereler arası 1 kare çakışma (önceki pencerenin BA-düzeltilmiş son karesi bir sonrakinin çapası olur).

2.4 — İlk Doğrulama Sonuçları

Kontrol	Sonuç
Derleme	Hatasız
Yerçekimi kestirimi	norm=9.81 (beklenen büyüklük)
BA başarı oranı (ilk test)	60/88 pencere (~%68)
NaN/Inf	Sıfır

Adım 3 — Build Taşınabilirliği ve Kamera Distorsiyonu

Dosyalar: CMakeLists.txt, src/main.cpp

CMakeLists.txt:

macOS'a özel sabit 'set(OpenCV_DIR /opt/homebrew/...)' satırı kaldırıldı (bu makinede böyle bir yol yok — kullanıcı tarafından zaten temizlenmişti). Kullanılmayan nlohmann_json bağımlılığı (koda hiç import edilmemişti) find_package ve target_link_libraries'den çıkarıldı.

Kamera distorsiyonu:

EuRoC MH01 cam0'ın gerçek radial-tangential distorsiyon katsayıları ($k_1, k_2, p_1, p_2 = -0.28340811, 0.07395907, 0.00019359, 1.76e-05$ — buildCameraMatrix() ile aynı kalibrasyon kaynağından) eklendi. initUndistortRectifyMap ile haritalar bir kez önceden hesaplanıp her karede cv::remap ile uygulanıyor — böylece feature çıkarımı, triangülasyon ve BA reprojection hatası artık doğru (distorsiyonsuz) piksel koordinatları üzerinden çalışıyor.

Performans İyileştirmeleri

ORB nfeatures: 10000 → 3000

BFMatcher maliyeti $O(N \cdot M)$ (karesel) olduğundan bu tek değişiklik eşleştirme maliyetini teoride $\sim 11x$ azaltır. Ölçülen gerçek etki:

	nfeatures=10000	nfeatures=3000
İşlenen kare (25s)	176	561
Yaklaşık FPS	~ 7.0	~ 22.4
BA başarı oranı	%56	%53 (neredeyse aynı)

Sonuç: $\sim 3.2x$ throughput artışı, iz-hayatta-kalma oranında endişe edilen kötüleşme gerçekleşmedi.

TimestampedDataSource::closestIndex — $O(N) \rightarrow O(\log N)$

Lineer arama yerine std::lower_bound ile ikili arama. Dürüstçe belirtilmeli: bu, ana işleme döngüsünde değil sadece program başlangıcında (GT hizalama) çalışıyor — gerçek etkisi küçük (birkaç milisaniye), ama ImuPreintegrator'ın zaten kullandığı yaklaşımla tutarlılık ve doğru asimptotik karmaşıklık için düzeltildi.

TrajectoryViewer::redrawAll() — $O(n^2)$ çizim throttling

autoFit=true iken her karede TÜM geçmiş yeniden çiziliyordu (800 karede toplam ~ 320.000 çizim operasyonu). Yeni autoFitInterval parametresi (varsayılan 10) ile tam yeniden-çizim sadece her 10 karede bir yapılıyor, aradaki kareler ucuz incremental ekleme kullanıyor. Dürüst not: ölçümde bu değişikliğin genel throughput'a gözle görülür bir etkisi olmadığı tespit edildi — çizim, gerçek darboğaz değilmiş; yine de doğru bir algoritmik iyileştirme olduğu ve uzun sekanslarda önem kazanacağı için korundu.

Kalıcı Zamanlama Teşhisi Eklendi

cv::getTickCount() tabanlı ölçümlerle remap+detect, match ve BA süreleri konsola yazdırılıyor (kalıcı, gelecekte ayar yaparken varsayımla değil ölçümle karar vermek için). Ölçülen ortalamalar: detect $\sim 9.5ms$, match $\sim 9.4ms$, BA $\sim 75.5ms$ /pencere ($\sim 15ms$ /kare amortize) — tek bir baskın darboğaz yok, dengeli bir dağılım.

Görselleştirme / ATE Hizalama Düzeltmeleri

Dosya: include/Test.hpp (TrajectoryViewer)

voRotationDeg = -75° kalıntı hilesi kaldırıldı

Bu parametre SADECE çizimi etkiliyordu, ATE hesabını hiç etkilemiyordu (computeLiveATE ham voPath kullanır). Adım 1'den ÖNCEKİ (kamera-merkezi hatası düzeltilmeden önceki) yanlış konumu gözle telafi etmeye çalışan bir kalıntıydı — matematik artık doğru olduğu için kaldırıldı.

alignVoForDraw devreye alındı (önceden yazılmış ama hiç çağrılmayan ölü kod)

computeLiveATE() VO'yu GT'ye Umeyama ile (rotasyon+ölçek+öteleme) hizalayıp düşük bir ATE hesaplıyordu, ama EKRANA çizilen VO bu hizalamayı hiç kullanmıyordu — sadece skaler bir 'yol uzunluğu oranı' ile GT büyütülüyordu, VO ham haliyle çiziliyordu. Kodda tam bunu çözecek alignedVoForDraw() fonksiyonu vardı ama hiçbir yerden çağrılmıyordu. redrawAll(), updateFitScale() ve incremental çizim yollarının hepsi artık bu hizalamayı (tek noktalık alignVoForDraw() haline getirilerek) kullanıyor — artık ekranda görünen ile ATE sayısı AYNI hizalamayı temsil ediyor.

ATE konsola da yazdırılıyor

Önceden sadece ekran-üstü metinde görünüyordu; artık [ATE] kare N: değer şeklinde konsola da basılıyor — trend analizi ve teşhis için gerekli oldu.

Kritik Hata Avı: Hız / Bias / Ölçek Patlaması

Kullanıcı 'ATE yüksek, VO ve GT çizgileri çok farklı görünüyor' diye bildirdi. Bu, oturumun en derin ve en önemli teşhis-düzeltilme döngüsüyü.

7.1 — İlk Gözlem

Görselleştirme düzeltildikten sonra bile 'GT görsel ölçek' (VO yolu / GT yolu oranı) 800 kare boyunca 0.05'ten 30.955'e kadar SÜREKLİ büyüyordu — ATE düşük görünse de (Umeyama ölçeği absorbe ediyor) bu, VO'nun ham çıktısının gerçek metre biriminde OLMADIĞINI gösteriyordu.

7.2 — Yanlış Çıkan İlk Hipotez (dürüstçe raporlanıyor)

İlk hipotez: reprojection residual sayısı (iz×kare×2) IMU residual sayısını (kare_çifti×15) sayıca >100x geçiyor, bu yüzden IMU ölçeği sabitleyemiyor. Denendi: Imufactor.hpp'deki ağırlıklar 10-20x artırıldı. Sonuç: BAŞARISIZ — ölçek patlaması 30.955x'ten 4.5 MİLYON x'e çıkarak KATMERLİ kötüleşti. Bu deney açıkça yanlışlandı ve ağırlıklar geri alındı.

7.3 — Gerçek Kök Neden (doğrudan ölçümle bulundu)

Her pencere sınırında çapanın |hız|, |konum|, |bias| büyüklükleri konsola yazdırılarak izlendi. Bulgu: Pencere 3'te iz kalmayınca (BA çalışmadı) sistem ham IMU tahminine düştü, hız 0.005→0.77 m/s sıçradı. Sonraki pencerelerde BA 'başarılı' dönmeye devam etse bile bu hızı düzeltmek yerine katmerli büyüttü: |v| 10.56 → 32.07 → 116 m/s'ye, |ba| (ivmeölçer bias) 0.9 → 7.56 → 1119 m/s²'ye kadar çıktı — fiziksel olarak imkansız bir değer (gerçek MEMS bias'ı tipik olarak <0.1 m/s²'dir). Kök sebep: ba/bg/vel için HİÇBİR mutlak sınır yoktu — sadece 'ardışık kareler arası çok değişmesin' (random-walk) kısıtı vardı.

7.4 — Kalıcı Çözüm: Mutlak Prior (Regularization)

Yeni BiasPriorError yapısı (Imufactor.hpp) — 3B bir vektörü ağırlıklandırılmış şekilde sıfıra çeken genel bir residual. Viobundleadjustment.hpp'de her pencerenin HER karesine uygulandı:

Parametre	Ağırlık	Yumuşak sınır (~1/ağırlık)
İvmeölçer bias (ba)	20.0	~0.05 m/s ²
Gyro bias (bg)	200.0	~0.005 rad/s
Hız (vel)	0.1	~10 m/s (çok gevşek — gerçek hareketi bastırmaz)

7.5 — Ölçülen Sonuçlar (800 karelik tam koşu, önce/sonra)

Metrik	Önce (hatalı)	Sonra (düzeltilmiş)
GT görsel ölçek	0.05 → 30.955 (patlıyor)	0.05 → 0.62 (stabil)
Hız v	0 → 2.49 milyon m/s	tüm koşu <0.25 m/s
İvmeölçer bias ba	0.9 → 1119 m/s ²	~0 (gerçekçi)
BA başarı oranı	~%53-68	%95.5 (190/199)

BA başarı oranındaki artış beklenmedik bir bonusu: sınırsız bias/hız değerleri optimizasyonun genel kararlılığını da bozuyormuş.

Güncel Durum ve Dürüst Değerlendirme

İlk testler nFrames=800 (EuRoC MH01'in sadece ilk ~40 saniyesi) ile yapılmıştı. Bu pencerede GT'nin kendisi sadece $0.47 \times 0.60 \times 0.58$ metrelik küçük bir alanda hareket ediyordu (kalkış sonrası durgun bölge) — bu yüzden ATE=0.13-0.16m 'iyi' gibi görünüyordu, ama GT'nin kapladığı alanın %25-30'u kadardı; görsel karşılaştırma bunu netçe ortaya koydu ('iki bağımsız çizgi gibi').

nFrames 2000'e çıkarılıp (~100 saniye, ~40m gerçek yol) yeniden test edildi. Sonuç: durgun bölgede (kare 0-1000) ATE düşük kalıyor, ama gerçek hareket başladığında (kare ~1000+) ATE hızla ~2 metreye çıkıp orada dengeleniyor (patlamıyor, ama önemli bir sürüklenme var). ~40m yola karşı ~2m hata $\approx$ %5 mesafe-oranlı sürüklenme.

Bulgu	Değerlendirme
Katastrofik patlama (hız/bias/ölçek)	Gerçekten düzeldi — doğrulanmış
Görselleştirme/ATE tutarlılığı	Gerçekten düzeldi — doğrulanmış
Durgun bölgede doğruluk	İyi
Gerçek harekette doğruluk	~%5 mesafe-oranlı sürüklenme — mimari bir sınırlama

Bu kalan sürüklenme, muhtemelen: (a) 5 karelik (~0.2sn) pencerelerin hız/ölçek için zayıf gözlemlenebilirlik sağlaması, (b) iz-hayatta-kalma oranının hâlâ tutarsız olması (ORB+BFFMatcher zincirleme, KLT değil), (c) global bir düzeltme mekanizmasının (loop closure/pose graph) hiç olmaması nedeniyle oluşuyor.

Adım 4 (Deneme) — KLT Optik Akış Tabanlı İzleme

Hipotez: '0 iz' pencerelerinin sebebi ORB+BFMatcher'ın kare-kare bağımsız eşleştirmesinin (4 sıçramalık zincirin herhangi bir halkası kopunca TÜM izin atılması) kırılğan olmasıdır. KLT (Lucas-Kanade optik akış), aynı noktayı sürekli id ile takip ederek bu zincirleme sorununu ortadan kaldırır.

Yapılanlar:

- Yeni include/pip05_KltTracker.hpp: goodFeaturesToTrack (Shi-Tomasi) + calcOpticalFlowPyrLK, RANSAC temel-matris tutarlılık filtresi, otomatik takviye (replenishment)
- main.cpp'nin izleme mantığı KLT'ye bağlandı, ORB+BFMatcher pipeline'dan çıkarıldı (dosyalar silinmedi)
- pip04_VioWindow.hpp'deki buildFullCoverageTracks silinip yerine id-tabanlı kesişim mantığı (buildWindowTracksFromKltHistory) kondu

Ölçülen Sonuç (aynı 2000 karelik test, ORB temeliyle karşılaştırma):

Metrik	ORB+BFMatcher (temel)	KLT (deneme)
Pencere başarı oranı	484/499 (%97)	434/499 (%87)
ATE (kare 2000)	1.98 m	2.24 m

Sonuç: BAŞARISIZ — KLT her iki metrikte de ORB'dan daha kötü çıktı. Ama önemli bir teşhis imkanı sağladı (aşağıya bakınız).

Kritik Teşhis: Asıl Sorun İzleme Değil, Triangülasyonmuş

'0 iz' pencerelerinden birinde (Pencere 3) geçici teşhis kodu eklenip incelendi: KLT aslında 160 tam-kapsamalı iz üretmişti (izleme başarılıydı!) — ama bu izlerin TAMAMI bir sonraki adımda, triangulateAndFilterWindowTracks'ın cheirality (derinlik-pozitifliği) filtresinde reddedildi. RANSAC tekli-sıçrama hayatta kalma oranı da sağlıklıydı (%85-99). Bu, orijinal hipotezi (kırık eşleşme zincirleri) YANLIŞLADI: asıl sebep, düşük-hareketli (neredeyse durgun) pencerelerde triangülasyonun sayısal olarak dejenere olması — kaç iz verilirse verilsin, kamera yeterince hareket etmemişse geometri elverişsiz kalıyor.

Karar:

KLT değişikliği geri alındı (kullanıcı tercihi de ORB yönündeydi). pip05_KltTracker.hpp dosyası silinmeden, kullanılmadan bırakıldı — ileride (loop-closure/relocalization gibi) işe yarayabilir.

Adım 5 (Deneme) — Çok-Görüşlü (N-view) Triangülasyon

Hipotez: triangulateAndFilterWindowTracks sadece pencerenin İLK ve SON karesini kullanıyordu (cv::triangulatePoints, 2-görüşlü) — ortadaki 3 karenin gözlem verisi tamamen israf ediliyordu. Standart çok-görüşlü DLT (Hartley&Zisserman; her görüşün 2 lineer denklemi tek bir $(2N \times 4)$ sisteme yığılıp SVD ile çözülür) pencerenin TÜM 5 karesini kullanarak düşük-baseline durumlarda bile daha iyi sayısal koşullanma sağlamalıydı.

Ölçülen Sonuç (aynı 2000 karelik test):

Metrik	2-görüşlü (temel)	N-görüşlü (deneme)
--------	-------------------	--------------------

Pencere başarı oranı	484/499 (%97)	481/499 (%96.4) — hemen hemen aynı
ATE (kare 2000)	1.98 m	4.01 m — 2 KAT DAHA KÖTÜ

Sonuç: BAŞARISIZ — pencere başarı oranı neredeyse değişmedi (beklenen iyileşme gerçekleşmedi), ATE ise belirgin şekilde kötüleşti. Olası açıklama: triangülasyon, BA'dan ÖNCE, henüz optimize edilmemiş IMU-seed'lenmiş pozlarla çalışıyor; pencerenin ortasındaki kareler (1,2,3) çapadan itibaren ham IMU dead-reckoning ile tahmin ediliyor, yani kendileri de gürültülü. Bu gürültülü ara pozları triangülasyona ek kısıt olarak eklemek, teorik kazancı pratikte gürültü olarak geri getirmiş olabilir.

Karar:

Değişiklik geri alındı, 2-görüşlü (kanıtlanmış, çalışan) triangülasyona dönüldü.

Genel Özet — Bugüne Kadar Denenen Tüm Yöntemler

#	Yöntem	Sonuç
1	Kamera merkezi formülü düzeltilmesi ($C = -R^T t$)	BAŞARILI — kalıcı
2	IMU/BA entegrasyonu + t_{wc}/C karışıklığı düzeltilmesi	BAŞARILI — kalıcı
3	IMU→kamera çerçeve rotasyonu + statik yerçekimi	BAŞARILI — kalıcı
4	Build taşınabilirliği + kamera distorsiyonu	BAŞARILI — kalıcı
5	ORB nfeatures 10000→3000, closestIndex $O(\log N)$, redrawAll throttling	BAŞARILI — kalıcı
6	voRotationDeg kaldırma + alignVoForDraw bağlama	BAŞARILI — kalıcı
7	IMU ağırlıklarını artırma (rotW/velW/posW 10-20x)	BAŞARISIZ — geri alındı
8	Bias/hız mutlak prior'u (BiasPriorError)	BAŞARILI — kalıcı
9	KLT optik akış tabanlı izleme	BAŞARISIZ — geri alındı
10	Çok-görüşlü (N-view) triangülasyon	BAŞARISIZ — geri alındı

Genel ders: 7, 9 ve 10 numaralı denemelerin hepsi teorik olarak makul hipotezlerdi, ama üçü de ölçüldüğünde işe yaramadı ya da kötüleştirdi. Bu, sistemin sorunlarının çoğunlukla 'yerel/parametrik' değil, mimari olduğuna işaret ediyor — sabit 5-karelik pencerenin kameranin gerçek hareketinden bağımsız olarak zorla kapatılması, altında yatan asıl kısıt.

Sonraki Adım: Keyframe / Paralaks Tabanlı Pencereleme

Sabit kare-sayısı yerine, pencereyi kameranin GERÇEKTEN yeterince hareket ettiği (paralaks eşiği aşıldığı) ana kadar açık tutmak. Bu, bugün doğrulanan kök nedeni (düşük-hareket pencerelerinde triangülasyonun geometrik olarak imkansız olması) doğrudan hedefleyen, mimari düzeyde bir çözüm. Detaylı tasarım ve uygulama sonraki oturumda ele alınacaktır.